Fuzzy Systems

A Simple Introduction

An easy, example-based guide for B.Tech students

Dr. Kanchan Rajwar

Department of Mathematics
Indian Institute of Information Technology, Bhopal

Optimization Techniques • CSE(DS)-5003 • Module 3

To the Student

This small book explains Fuzzy Systems in simple words. It is written for students who are meeting the topic for the first time. Every idea is built slowly — first we see why the idea is needed, then we learn the idea, then we work through an example. Short sentences are used throughout. You do not need advanced mathematics. You only need basic algebra.

Read the chapters in order. Try the examples with pen and paper as you go. By the end, you will be able to build a simple fuzzy system by hand and answer exam questions with confidence.

How to use these notes:

1. Attend the class lecture for each topic.
2. Read the corresponding section of these notes carefully, working through each example yourself.
3. Once you have finished a chapter, attempt the related questions in the companion question bank (*Fuzzy Systems — Question Bank*) for self-assessment.
4. Check your answers against the answer key only *after* attempting the questions on your own.

The question bank contains 60 multiple choice questions, 5 short answer questions, and 5 broad questions — all based directly on these notes and the running example developed in class. Regular self-assessment is one of the most effective ways to prepare for exams. Do not skip it.

Good luck, and enjoy the topic.

Contents

Preface

This book is a module-level introduction to Fuzzy Systems, prepared for students of Optimization Techniques (CSE(DS)-5003) at the Indian Institute of Information Technology, Bhopal. It forms part of Module 3 of the course, alongside the companion notes on Genetic Algorithms.

The notes are written to be self-contained. A student who reads carefully, works through every example, and attempts the companion question bank should be fully prepared for examination questions on this topic.

The running example — assessing student academic performance using fuzzy sets (Poor, Average, Good) — appears in every chapter. This consistency is deliberate: it allows the reader to see exactly how each concept connects to the next, making it easier to understand the system as a whole rather than as a collection of isolated ideas.

Comments and suggestions for improvement are welcome.

Dr. Kanchan Rajwar
Department of Mathematics
Indian Institute of Information Technology, Bhopal
August 2026

1. Introduction to Fuzzy Systems

1.1 What is Fuzzy Logic?

In everyday life, many things are not simply true or false. A student is not just "good" or "bad" — they may be "somewhat good" or "fairly average". Classical logic forces every statement into one of two categories: 0 (false) or 1 (true). Fuzzy logic allows any value between 0 and 1, capturing the idea of *partial truth*.

This idea was introduced by Lotfi Zadeh in 1965. It gives us a mathematical way to handle vague, imprecise, or uncertain information — the kind of information humans use every day.

> *Classical logic:* A student with marks ≥ 60 is good. A student with marks < 60 is not good.
>
> *Fuzzy logic:* A student with marks 59 is *almost* good. A student with marks 45 is *somewhat* good.

1.2 Why Classical Logic is Not Always Enough

Classical (crisp) logic works well for precise, well-defined problems. But many real-world problems involve vagueness and uncertainty.

- **Sharp boundaries are unrealistic.** A student with 59 marks is nearly identical to one with 60, yet classical logic treats them completely differently.
- **Human reasoning is naturally fuzzy.** When a teacher says a student is "fairly good", they do not mean exactly 72.5 marks. They mean somewhere in a range, with varying degrees of "goodness".
- **Many systems need gradual control.** A thermostat that switches sharply between ON and OFF is less efficient than one that adjusts gradually based on how hot or cold the room is.

Fuzzy logic addresses all of these situations by allowing gradual, partial membership rather than sharp yes/no boundaries.

1.3 Running Example

Throughout this book we use the following problem.

Problem. A university wants to assess student academic performance based on their marks (out of 100). The categories are: *Poor*, *Average*, and *Good*. Instead of hard cutoffs, the university wants a smooth, gradual assessment that reflects how students actually perform.

We will build a complete fuzzy system for this problem, step by step, across all chapters.

1.4 Applications of Fuzzy Systems

Fuzzy systems are used in a wide range of real-world applications.

- **Consumer electronics:** washing machines, air conditioners, cameras with fuzzy auto-focus
- **Control systems:** train braking systems, industrial process control
- **Medical diagnosis:** assessing disease severity from imprecise symptoms
- **Decision support:** credit scoring, student assessment, risk evaluation
- **Robotics:** navigation and obstacle avoidance under uncertainty

Summary

- Classical logic assigns 0 or 1 to every statement. Fuzzy logic allows any value in $[0, 1]$.
- Fuzzy logic captures partial truth and gradual membership — closer to how humans reason.
- Introduced by Lotfi Zadeh in 1965.
- Used in electronics, control, medicine, finance, and robotics.

2. Fuzzy Set Theory

2.1 Classical Sets vs. Fuzzy Sets

In a **classical set**, an element either belongs to the set or it does not. There is no middle ground.

Example 2.1. Define the classical set $G =$ "Good students" as those with marks ≥ 60.

$$G = \{x \mid x \geq 60\}$$

A student with marks 60 is in G (membership $= 1$). A student with marks 59 is not in G (membership $= 0$). The jump from 59 to 60 causes a sharp, unrealistic boundary.

A **fuzzy set** removes this sharp boundary. Instead of membership being 0 or 1, it can be any value in $[0, 1]$. A student with marks 59 might have membership 0.97 in the Good set — almost good, but not quite.

2.2 Membership Function

The **membership function** $\mu_A(x)$ tells us how much an element x belongs to a fuzzy set A. It maps each value of x to a number in $[0, 1]$.

$$\mu_A : X \to [0, 1]$$

- $\mu_A(x) = 1$: x fully belongs to A
- $\mu_A(x) = 0$: x does not belong to A
- $\mu_A(x) = 0.6$: x partially belongs to A (60% membership)

2.3 Triangular Membership Function

The **triangular membership function** is the simplest and most widely used shape. It is defined by three points $(a,\ b,\ c)$ where $a \leq b \leq c$:

$$\mu(x; a, b, c) = \begin{cases} 0 & x \leq a \\ \dfrac{x-a}{b-a} & a < x \leq b \\ \dfrac{c-x}{c-b} & b < x \leq c \\ 0 & x > c \end{cases}$$

Here a is the left foot (membership rises from 0), b is the peak (membership $= 1$), and c is the right foot (membership falls back to 0).

Example 2.2. Define three fuzzy sets for our running example over marks $x \in [0, 100]$:

Fuzzy Set	a	b	c	Meaning
Poor	0	0	50	*Lowmarks*
Average	30	50	70	*Middlemarks*
Good	50	100	100	*Highmarks*

For a student with marks $x = 40$:

$$\mu_{\text{Poor}}(40) = \frac{50 - 40}{50 - 0} = \frac{10}{50} = 0.20$$

$$\mu_{\text{Average}}(40) = \frac{40 - 30}{50 - 30} = \frac{10}{20} = 0.50$$

$$\mu_{\text{Good}}(40) = 0 \quad (\text{since } 40 < 50)$$

So a student with marks 40 is 20% Poor, 50% Average, and 0% Good. This is far more informative than simply saying the student "failed".

2.4 Types of Membership Functions

2.4.1 Trapezoidal Membership Function

Defined by four points (a, b, c, d). Has a flat top (full membership = 1) between b and c.

$$\mu(x; a, b, c, d) = \begin{cases} 0 & x \le a \\ \dfrac{x - a}{b - a} & a < x \le b \\ 1 & b < x \le c \\ \dfrac{d - x}{d - c} & c < x \le d \\ 0 & x > d \end{cases}$$

Useful when a range of values all fully belong to the set.

Example 2.3. For the Good set with a trapezoidal function ($a = 60, b = 75, c = 90, d = 100$): any student with marks between 75 and 90 has membership = 1 (fully Good), while those between 60–75 and 90–100 have partial membership.

2.4.2 Gaussian Membership Function

A smooth bell-shaped curve defined by centre c and width σ:

$$\mu(x) = e^{-\frac{(x-c)^2}{2\sigma^2}}$$

The peak is at $x = c$ with membership = 1. Membership decreases smoothly in both directions. Suitable when a gradual, smooth transition is needed.

2.4.3 Singleton Membership Function

Membership is 1 at exactly one point and 0 everywhere else. Used mainly in the output (consequent) of fuzzy rules.

2.5 Fuzzy Set Operations

Just as classical sets have union, intersection, and complement, fuzzy sets have analogous operations. Let A and B be two fuzzy sets.

Union (OR) — the larger membership wins:

$$\mu_{A\cup B}(x) = \max\big(\mu_A(x),\ \mu_B(x)\big)$$

Intersection (AND) — the smaller membership wins:

$$\mu_{A\cap B}(x) = \min\big(\mu_A(x),\ \mu_B(x)\big)$$

Complement (NOT):

$$\mu_{\bar{A}}(x) = 1 - \mu_A(x)$$

Example 2.4. For a student with marks $x = 40$:

$$\mu_{\text{Poor}}(40) = 0.20, \qquad \mu_{\text{Average}}(40) = 0.50$$

Union (Poor OR Average):

$$\mu_{\text{Poor}\cup\text{Average}}(40) = \max(0.20,\ 0.50) = 0.50$$

Intersection (Poor AND Average):

$$\mu_{\text{Poor}\cap\text{Average}}(40) = \min(0.20,\ 0.50) = 0.20$$

Complement (NOT Poor):

$$\mu_{\overline{\text{Poor}}}(40) = 1 - 0.20 = 0.80$$

The complement tells us the student is 80% "not poor" — which matches our intuition for marks of 40.

2.6 Linguistic Variables

A **linguistic variable** is a variable whose values are words rather than numbers.

In our running example:

- **Variable name:** Academic Performance
- **Domain (universe of discourse):** $[0, 100]$ (marks out of 100)
- **Linguistic values:** Poor, Average, Good
- **Each linguistic value** is a fuzzy set with its own membership function

Linguistic variables allow us to describe and reason about imprecise concepts — like "good performance" — in a mathematically precise way.

Summary

- A **fuzzy set** assigns each element a membership value in $[0, 1]$.
- The **membership function** $\mu_A(x)$ measures how much x belongs to A.
- Common shapes: triangular (a, b, c), trapezoidal (a, b, c, d), Gaussian.
- Fuzzy operations: union = max, intersection = min, complement = $1 - \mu$.
- A **linguistic variable** takes words as values, each represented by a fuzzy set.

3. Fuzzy Inference System

3.1 What is a Fuzzy Inference System?

A **Fuzzy Inference System** (FIS) is a system that uses fuzzy logic to map a crisp input to a crisp output. It mimics human reasoning by using if-then rules with fuzzy sets instead of precise numerical conditions.

A FIS has four steps:

Step 1. Fuzzification: Convert the crisp input into fuzzy membership values.

Step 2. Rule Evaluation: Apply fuzzy if-then rules to the membership values.

Step 3. Aggregation: Combine the outputs of all rules into one fuzzy set.

Step 4. Defuzzification: Convert the combined fuzzy output into a single crisp value.

We trace all four steps using the running example.

3.2 Step 1 — Fuzzification

Fuzzification takes a crisp input value and computes its degree of membership in each fuzzy set. This is simply the evaluation of each membership function at the input value.

Example 3.1. For a student with marks $x = 40$, using the triangular membership functions defined in Chapter 2:

$$\mu_{\text{Poor}}(40) = 0.20, \quad \mu_{\text{Average}}(40) = 0.50, \quad \mu_{\text{Good}}(40) = 0.00$$

The student is 20% Poor, 50% Average, and 0% Good. These three numbers are the input to the rule evaluation step.

3.3 Step 2 — Fuzzy Rules

Fuzzy if-then rules express knowledge in linguistic terms. Each rule has:

- An **antecedent** (IF part): a condition stated in terms of the input fuzzy sets
- A **consequent** (THEN part): a conclusion stated in terms of the output fuzzy sets

For our example, the output variable is *Grade*, assessed over a grade point scale $[0, 100]$. Just as the input variable (Academic Performance) was described by three fuzzy sets (Poor, Average, Good), the output variable is described by three fuzzy sets (Fail, Pass, Distinction). Each output set is represented as a **singleton membership function** — it has membership = 1 at exactly one point (its peak) and 0 everywhere else.

Output Set	Peak point b	Meaning
Fail	20	Poor performance
Pass	50	Satisfactory performance
Distinction	90	Excellent performance

These peak points are chosen by the domain expert — the same way the input membership function parameters (a, b, c) were chosen in Chapter 2. The points 20, 50, and 90 represent the "centre" of each grade category on the $[0, 100]$ scale. How these output sets are used in rule evaluation and defuzzification is explained in Steps 3 and 4 below.

Rule base:

R1. IF performance is *Poor* THEN grade is *Fail*

R2. IF performance is *Average* THEN grade is *Pass*

R3. IF performance is *Good* THEN grade is *Distinction*

3.4 Step 3 — Rule Evaluation and Aggregation

Each rule fires with a **firing strength** equal to the membership value of its antecedent. The output fuzzy set of each rule is *clipped* (cut) at its firing strength. This is the **Mamdani method**.

All clipped output sets are then combined using the union (max) operation — this is called **aggregation**.

Example 3.2. For marks $x = 40$:

R1 (Poor → Fail): fires at 0.20 ⇒ Fail clipped at 0.20

R2 (Average → Pass): fires at 0.50 ⇒ Pass clipped at 0.50

R3 (Good → Distinction): fires at 0.00 ⇒ Distinction not activated

The aggregated output is the union of all clipped sets. In our discrete approximation:

Output point	Set	μ
20	Fail	0.20
50	Pass	0.50
90	Distinction	0.00

3.5 Step 4 — Defuzzification

Defuzzification converts the aggregated fuzzy output into a single crisp value. The most common method is the **centroid method** (centre of gravity):

$$y^* = \frac{\sum_i y_i \cdot \mu(y_i)}{\sum_i \mu(y_i)}$$

Example 3.3. Using the aggregated output from Step 3:

$$y^* = \frac{20 \times 0.20 + 50 \times 0.50 + 90 \times 0.00}{0.20 + 0.50 + 0.00} = \frac{4 + 25 + 0}{0.70} = \frac{29}{0.70} \approx 41.43$$

The crisp output grade point is approximately 41.43. To understand why, recall the firing strengths that produced this output:

Rule	Output set (peak)	Firing strength
R1: IF Poor THEN Fail	Fail (20)	0.20
R2: IF Average THEN Pass	Pass (50)	0.50
R3: IF Good THEN Distinction	Distinction (90)	0.00

The centroid is a weighted average. Pass (50) carries the highest weight (0.50), pulling the result strongly toward 50. Fail (20) carries a smaller weight (0.20), pulling the result slightly toward 20. Distinction (90) has zero weight and contributes nothing. The result 41.43 lies between 20 and 50, closer to 50.

Therefore, the student is **predominantly Average with a slight Poor component** — which matches our intuition for marks of 40.

3.6 Other Defuzzification Methods

Mean of Maxima (MOM): Takes the average of all output values that achieve the maximum membership in the aggregated set. Simple but ignores the shape of the output set.

Smallest of Maxima (SOM): Takes the smallest output value that achieves the maximum membership. Used when a conservative (safe) decision is preferred.

Largest of Maxima (LOM): Takes the largest output value at maximum membership. Used when an aggressive decision is preferred.

For most applications, the centroid method is preferred because it considers the full shape of the output set.

Summary

- A **FIS** maps a crisp input to a crisp output using four steps.
- **Fuzzification:** crisp input → membership values using membership functions.
- **Rules:** IF condition THEN conclusion; each rule fires at the membership strength of its antecedent.
- **Aggregation:** combine clipped rule outputs using max.
- **Defuzzification:** aggregated fuzzy set → crisp output using the centroid: $y^* = \sum y_i \mu_i / \sum \mu_i$.

4. Optimization of Fuzzy Systems

4.1 Why Optimize a Fuzzy System?

A fuzzy system has many design choices:

- The shape and parameters of each membership function (e.g. the values a, b, c of each triangle)
- The number and form of the fuzzy rules
- The defuzzification method

Choosing these by hand is difficult and subjective. Different designers may produce very different systems for the same problem. **Optimization of fuzzy systems** means using an automatic method to find the best design choices. A Genetic Algorithm is one of the most widely used tools for this purpose.

4.2 What Do We Optimize?

The most common optimization target is the **membership function parameters**. Given a dataset of known input-output pairs, we search for the values of a, b, c that make the fuzzy system's output as close as possible to the known correct output.

Example 4.1. In our running example, suppose we have the following known assessments from a faculty panel:

Student	Marks x	Desired grade point y
A	40	42
B	55	55
C	70	73
D	85	88

We want to find membership function parameters that make the FIS output match these desired values as closely as possible.

The objective is to minimise the Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

where y_i is the desired output and $\hat{y}_i$ is the FIS output for student i.

4.3 Using a GA to Optimize a Fuzzy System

We now use a GA to search for the best membership function parameters. Each step maps directly to the GA concepts in the companion notes.

Step 1. Encode the parameters as a chromosome. Each chromosome encodes the nine parameters (a, b, c) for the three fuzzy sets (Poor, Average, Good):

$$\text{Chromosome} = [a_P,\ b_P,\ c_P,\ a_A,\ b_A,\ c_A,\ a_G,\ b_G,\ c_G]$$

Real-coded encoding is used since the parameters are continuous values in $[0, 100]$.

Step 2. Define the fitness function. For each chromosome, run the FIS on all four training students and compute the MSE. Since the GA maximises fitness and we want to minimise MSE, we transform:

$$F = \frac{1}{1 + \text{MSE}}$$

A perfect fit (MSE $= 0$) gives fitness $F = 1$. A poor fit gives fitness close to 0.

Step 3. Run the GA. Initialise a random population of parameter sets. Apply selection, crossover, and mutation to evolve toward lower MSE. After T generations, the best chromosome gives the optimised membership function parameters.

Example 4.2. **Initial parameters** (hand-chosen):

Set	a	b	c
Poor	0	0	50
Average	30	50	70
Good	50	100	100

Running the FIS on all four students gives:

Student	y_i	$\hat{y}_i$	$(y_i - \hat{y}_i)^2$
$A(x = 40)$	42	41.43	0.32
$B(x = 55)$	55	52.10	8.41
$C(x = 70)$	73	69.50	12.25
$D(x = 85)$	88	83.20	23.04

$$\text{MSE} = \frac{0.32 + 8.41 + 12.25 + 23.04}{4} = \frac{44.02}{4} = 11.01$$

$$F = \frac{1}{1 + 11.01} = 0.083$$

After 50 generations, the GA finds optimised parameters:

Set	a	b	c
Poor	0	0	46
Average	28	52	69
Good	54	100	100

$$\text{MSE} = 0.74, \qquad F = \frac{1}{1 + 0.74} = 0.575$$

The fitness improved from 0.083 to 0.575. The membership boundaries have shifted slightly to better match the faculty panel's assessments. The Poor set narrows (peak moves from 50 to 46), the Average set shifts slightly right (30–70 becomes 28–69), and the Good set tightens (50–100 becomes 54–100).

4.4 Other Aspects That Can Be Optimized

Rule base optimization. The set of if-then rules can also be evolved by a GA. Each chromosome encodes a complete rule base, and fitness measures how well the resulting FIS performs.

Number of fuzzy sets. The GA can search over different numbers of linguistic values (e.g. 3 vs. 5 sets) to find the best trade-off between simplicity and accuracy.

Defuzzification method. Different methods (centroid, MOM, SOM) can be compared and selected automatically.

4.5 Advantages of GA-Optimized Fuzzy Systems

- No gradient needed — the fitness function can be any measure of system performance.
- Works even when the relationship between parameters and performance is complex and non-linear.
- Can optimize membership functions, rules, and structure simultaneously.
- Results are interpretable — the optimized system is still a fuzzy system with readable rules.

4.6 Limitations

- Computationally expensive — each fitness evaluation requires running the full FIS on all training data.
- Large search space when both rules and parameters are optimized together.
- Risk of overfitting if the training dataset is small.
- No guarantee of finding the globally optimal parameters.

Summary

- Fuzzy systems have design parameters that are difficult to set by hand.
- A GA searches for the best parameters by minimising output error (MSE) on training data.
- **Chromosome:** encodes the membership function parameters.
- **Fitness:** $F = 1/(1 + \text{MSE})$ — higher is better.
- GA-optimized fuzzy systems combine the interpretability of fuzzy logic with the search power of evolutionary computation.

5. Advantages, Limitations, and Conclusion

5.1 Advantages of Fuzzy Systems

Handles imprecision naturally. Fuzzy systems are designed for vague, uncertain, and imprecise information. They do not require exact numerical inputs or outputs.

Close to human reasoning. Fuzzy rules are written in natural language (IF performance is Average THEN grade is Pass), making them easy to understand and verify by domain experts.

Smooth control. Unlike classical on/off systems, fuzzy systems produce gradual, smooth outputs that avoid sudden jumps.

No mathematical model required. A fuzzy system can be built from expert knowledge alone, without needing a precise mathematical model of the system being controlled.

Robust to noise. Because fuzzy systems deal with ranges and partial membership rather than exact values, they are naturally tolerant of noisy or imprecise input data.

5.2 Limitations of Fuzzy Systems

Membership function design is subjective. Choosing the shape and parameters of membership functions requires expertise and judgement. Different designers may make different choices.

No learning from data (by default). A basic fuzzy system does not learn or adapt. Rules and membership functions must be set manually — or optimized using a GA or other technique.

Rule explosion. As the number of input variables increases, the number of rules grows exponentially. A system with 5 inputs and 5 linguistic values per input requires up to $5^5 = 3125$ rules.

No guarantee of optimality. A hand-designed fuzzy system may not be the best possible system for the task.

5.3 Conclusion

Fuzzy Systems give us a principled way to handle the vagueness and imprecision that are present in almost every real-world problem. By replacing sharp yes/no boundaries with smooth, gradual membership functions, and by reasoning with natural-language rules, fuzzy systems produce decisions that are both mathematically sound and intuitively meaningful.

When combined with optimization — particularly Genetic Algorithms — fuzzy systems become even more powerful. The GA removes the burden of manually tuning membership functions and rules, automatically finding the design that best fits the available data. Together, fuzzy logic and evolutionary computation form a complementary pair: fuzzy logic provides the framework for human-like reasoning; the GA provides the engine for automatic improvement.

Summary

- Fuzzy systems handle imprecision, are close to human reasoning, and produce smooth outputs.

- Limitations: subjective design, no default learning, rule explosion for many inputs.
- GA optimization removes the need to tune membership functions by hand.
- Fuzzy logic + GA = interpretable system + automatic optimization.

Python Implementation

A step-by-step Python implementation of a Fuzzy Inference System, using the same running example (academic performance assessment) developed in these notes, is available as a Google Colab notebook on GitHub. No installation is required.

`github.com/kanchan999/Optimization-Techniques-CSE5003`

Note: After completing these notes, proceed to the companion Question Bank for self-assessment.

Best wishes.
